

Preguntas y Respuestas sobre Ingeniería de Software y Patrones de Diseño

1. **¿Cuál es la diferencia fundamental entre validación y verificación?**

La validación responde a la pregunta "*¿Estamos construyendo el producto correcto?*" y se enfoca en asegurar que el sistema satisface las expectativas y necesidades del cliente. La verificación responde a "*¿Estamos construyendo el producto correctamente?*" y se enfoca en comprobar que el software cumple estrictamente con su especificación técnica (requisitos funcionales y no funcionales).

2. **En la trazabilidad del Proceso Unificado, ¿qué describen las postcondiciones de un contrato de operación?**

Describen los cambios de estado que ocurren en los objetos del modelo del dominio después de que la operación se ha ejecutado. Esto incluye la creación o destrucción de instancias, la modificación de atributos y la formación o ruptura de asociaciones entre objetos.

3. **¿Qué criterios propone el Proceso Unificado para seleccionar los requisitos que se implementarán en las primeras iteraciones?**

Se basa principalmente en tres criterios: alto riesgo (para mitigarlos lo antes posible), alto valor de negocio (para el cliente) y significancia arquitectónica (que ayuden a definir la estructura del sistema).

4. **¿Qué tipo de requisitos se implementan en las primeras iteraciones del Proceso Unificado?**

Se implementan aquellos casos de uso y requerimientos críticos que atacan los mayores riesgos del proyecto y permiten establecer y estabilizar una arquitectura base ejecutable.

5. **¿Cuál es la ventaja de la inspección de software respecto de las pruebas?**

Existen tres ventajas fundamentales:

- Al ser un proceso estático, los errores no se enmascaran ni se ocultan entre sí, permitiendo encontrar múltiples defectos en una sesión.
- Se pueden inspeccionar versiones incompletas o representaciones legibles sin los costos adicionales de preparar el software para pruebas dinámicas.
- Permiten evaluar atributos de calidad más amplios, como la ineficiencia, mantenibilidad, portabilidad y el cumplimiento de estándares de código.

6. **¿En qué consiste la técnica de pruebas de particiones de entrada?**

Consiste en identificar particiones (grupos) de entrada y salida, y luego diseñar casos de prueba para que el sistema ejecute entradas de todas las particiones representativas y genere salidas en todas ellas, reduciendo drásticamente la cantidad de pruebas necesarias.

7. **¿Cuáles son los grupos o clases de equivalencia utilizados en la técnica de particiones de entrada?**

Son grupos de datos que comparten características o comportamientos comunes ante el

sistema. Por ejemplo: todos los números negativos, todos los nombres con menos de 30 caracteres, o todos los eventos provocados por la elección de un mismo menú.

8. **¿Por qué se eligió el patrón Observer en el caso de estudio del Sistema de Gestión Meteorológica?**

Se elige porque este patrón sirve para notificar automáticamente los cambios de estado de un objeto (como los nuevos datos del clima) a todos sus dependientes (como múltiples pantallas y vistas). Permite que las vistas se actualicen sin estar fuertemente acopladas a la fuente de datos.

9. **¿Cuál es la diferencia entre mantenimiento correctivo y mantenimiento adaptativo?**

El **mantenimiento correctivo** se encarga de diagnosticar y reparar los defectos o errores encontrados en el software. El **mantenimiento adaptativo** consiste en modificar el software para que se siga ejecutando adecuadamente ante cambios en su entorno operativo (nuevos sistemas operativos, bases de datos, hardware o leyes).

10. **¿Qué es el mantenimiento perfectivo?**

Es el mantenimiento que se realiza para agregar nuevos requisitos solicitados por el usuario o para realizar mejoras generales en la funcionalidad, el rendimiento o la facilidad de mantenimiento del software.

11. **¿Cuál es el propósito del patrón GOF Fachada (Facade)?**

Su propósito es proporcionar una interfaz unificada, más sencilla y de alto nivel a un conjunto de interfaces más complejas en un subsistema. Oculta la complejidad interna y facilita el uso del subsistema por parte de los clientes.

12. **¿Qué es una CMDB?**

Una CMDB (Configuration Management Database) es una base de datos que pretende tener un modelo de datos común para conseguir una mejor gestión de los servicios empresariales y elementos de TI.

13. **¿Cuál es el propósito u objetivo de una CMDB?**

Su objetivo es servir como una sola fuente confiable de información mediante la consolidación de los datos. Permite integrar procesos (como el manejo de incidentes y el manejo de cambios) reduciendo los costos administrativos y minimizando los errores.

14. **¿Qué objetivo tiene la fase de Elaboración del Proceso Unificado?**

Su objetivo principal es establecer una arquitectura base sólida, mitigar los riesgos más significativos descubiertos en la fase de Inicio, y capturar y detallar la gran mayoría de los requisitos del sistema.

15. **¿Cuándo es necesario aplicar el patrón Composite?**

Se aplica cuando se necesita representar jerarquías de objetos tipo "parte-todo" y se requiere que los clientes traten de manera idéntica y uniforme tanto a los objetos individuales (nodos simples) como a las composiciones de objetos (ramas).

16. **¿Qué relación de composición existe entre un Composite y sus componentes?**

Existe una relación de *agregación* o *composición* de 1 a muchos. El objeto compuesto

(Composite) contiene e itera sobre una colección de objetos *Componentes*, los cuales pueden ser nodos finales (Leaf) u otros Composite.

17. **¿Qué es un plan de retirada dentro de la Gestión de Cambios?**

También conocido como plan de "back-out", es un procedimiento de contingencia que permite deshacer o revertir un cambio para restaurar el sistema a su estado anterior en caso de que ocurra un funcionamiento incorrecto tras la implementación.

18. **¿Cuál es el objetivo de la Gestión de Cambios?**

Debe trabajar para asegurar que todos los cambios estén justificados, se lleven a cabo sin perjuicio de la calidad del servicio de TI, se registren, clasifiquen y documenten convenientemente, además de haber sido cuidadosamente testeados y reflejados en la CMDB.

19. **¿Qué muestra un diagrama de secuencia?**

Muestra las interacciones y el comportamiento dinámico del sistema; específicamente, ilustra la secuencia de los mensajes que son enviados entre objetos en el tiempo para responder a la invocación de un método o un caso de uso.

20. **¿Qué busca derivar la ingeniería inversa dentro de un proceso de reingeniería?**

Busca extraer información de diseño directamente a partir del código fuente existente de un sistema antiguo. Su nivel de abstracción ideal permite inferir modelos procedimentales, estructuras de datos o diagramas de relación de entidades.

21. **¿Qué establece la Ley de Complejidad Creciente de Lehman?**

Establece que a medida que un sistema de software se cambia y evoluciona, su estructura se degrada inherentemente, por lo que se debe hacer mantenimiento preventivo si se desea evitar que su complejidad aumente.

22. **¿Qué es un Elemento de Configuración (CI)?**

Es cualquier componente físico o lógico, de infraestructura, documentación o personal, que requiera ser registrado y gestionado estrictamente para asegurar la correcta provisión de un servicio de TI.

23. **¿Cuál es la diferencia entre un CI lógico y un CI físico?**

Un CI físico es tangible (un servidor, una impresora, un switch, un cableado). Un CI lógico es intangible pero fundamental para el servicio (una licencia de software, un acuerdo de nivel de servicio SLA, un proceso, una instancia de base de datos).

24. **¿Cuál es la diferencia entre el patrón State y el patrón Strategy?**

Aunque a nivel de código estructural se vean iguales, difieren en su intención. El patrón **State** permite que un objeto cambie su comportamiento dinámicamente porque su *estado interno* ha cambiado, haciendo parecer que cambia de clase. El patrón **Strategy** simplemente se usa para poder intercambiar libremente un *algoritmo* o tarea sin importar el estado del objeto.

25. **¿En qué se enfoca el patrón State?**

Se enfoca explícitamente en encapsular el estado interno de un objeto dentro de clases

dedicadas, de manera que el comportamiento de dicho objeto dependa completamente del estado en el que se encuentre.

26. ¿En qué se enfoca el patrón Strategy?

Se enfoca en encapsular un algoritmo (o familia de algoritmos) dentro de un objeto intercambiable, para que el algoritmo pueda variar independientemente del cliente o contexto que lo utilice.